

Systemctl and Linux Services

Service Management – Systemctl and Linux Services

Objective

To understand Linux service management using the `systemctl` command and learn how to start, stop, restart, enable, disable, monitor, and troubleshoot system services.

Introduction

In Linux, a **service** (also called a daemon) is a background process that runs continuously and provides specific functionality to the system.

Examples of Common Services

- SSH Service (`sshd`) – Remote login access
- Apache/Nginx – Web server services
- MySQL/MariaDB – Database services
- Docker – Container management service
- Cron – Task scheduling service

Modern Linux distributions use **systemd** as the service manager, and the primary command used to manage services is **systemctl**.

Basic Service Management Commands

1. Check Service Status

Display the current status of a service.

```
systemctl status ssh
```

or

```
systemctl status apache2
```

Possible states:

- `active (running)` – Service is running
- `inactive (dead)` – Service is stopped

- `failed` – Service encountered an error
-

2. Start a Service

Start a service that is currently stopped.

```
sudo systemctl start ssh
```

Verify:

```
systemctl status ssh
```

3. Stop a Service

Stop a running service.

```
sudo systemctl stop ssh
```

Verify:

```
systemctl status ssh
```

4. Restart a Service

Restart a service after configuration changes or troubleshooting.

```
sudo systemctl restart ssh
```

5. Reload a Service

Reload the service configuration without completely restarting the service.

```
sudo systemctl reload ssh
```

Difference Between Restart and Reload

Restart	Reload
Stops and starts the service again	Reloads configuration only
Causes temporary interruption	Usually no interruption

6. Enable a Service

Configure a service to start automatically during system boot.

```
sudo systemctl enable ssh
```

Verify:

```
systemctl is-enabled ssh
```

Expected Output:

```
enabled
```

7. Disable a Service

Prevent a service from starting automatically at boot.

```
sudo systemctl disable ssh
```

Verify:

```
systemctl is-enabled ssh
```

Expected Output:

```
disabled
```

8. List Running Services

Display all active services.

```
systemctl list-units --type=service
```

9. Check Failed Services

Display services that have failed.

```
systemctl --failed
```

10. View Service Logs

```
sudo nano /etc/ssh/sshd_config #Reference only
```

View logs generated by a specific service.

```
journalctl -u ssh
```

Show the last 20 log entries:

```
journalctl -u ssh -n 20
```

Monitor logs in real time:

```
journalctl -u ssh -f
```

Apache Service Management

Step 1: Install Apache Web Server

Ubuntu:

```
sudo apt update  
sudo apt install apache2 -y
```

Step 2: Check Apache Status

```
systemctl status apache2
```

Step 3: Stop Apache Service

```
sudo systemctl stop apache2
```

Open a browser and visit:

```
http://localhost
```

The webpage should no longer be accessible.

Step 4: Start Apache Service

```
sudo systemctl start apache2
```

Refresh the browser.

The Apache default webpage should now be accessible.

Step 5: Enable Apache at Boot

```
sudo systemctl enable apache2
```

Apache will automatically start whenever the system boots.

Real-World Troubleshooting Scenario

Problem:

A user reports that a website hosted on the server is not accessible.

Investigation Steps:

Check the Apache service status:

```
systemctl status apache2
```

If the service is stopped:

```
sudo systemctl start apache2
```

Check service logs:

```
journalctl -u apache2
```

Analyze error messages and take corrective action.

Assignment

Task 1

Check the status of the following services:

```
systemctl status ssh  
systemctl status cron  
systemctl status apache2
```

Record the status of each service.

Task 2

Perform the following operations on the Apache service:

- Start the service
- Stop the service
- Restart the service
- Check the status
- Enable the service
- Disable the service

Document each command used.

Task 3

Identify failed services on your system:

```
systemctl --failed
```

Take a screenshot of the output and include it in your submission.

Task 4

View the last 20 log entries for Apache:

```
journalctl -u apache2 -n 20
```

Save the output and explain any important log messages observed.

Creating and Troubleshooting a Failed Service

Objective

Learn how to identify, analyze, and fix a failed Linux service using Apache and systemctl.

Step 1: Check Apache Service Status

Verify that the Apache service is running.

```
systemctl status apache2
```

Expected Output:

```
Active: active (running)
```

Step 2: Create a Backup of the Configuration File

Before making any changes, create a backup.

```
sudo cp /etc/apache2/apache2.conf /etc/apache2/apache2.conf.bak
```

Step 3: Introduce a Configuration Error

Open the Apache configuration file.

```
sudo nano /etc/apache2/apache2.conf
```

Add the following invalid directive at the end of the file:

```
InvalidDirective On
```

Save and exit the editor.

Step 4: Validate the Configuration

Check the configuration syntax.

```
sudo apachectl configtest
```

Expected Output:

```
Syntax error
Invalid command 'InvalidDirective'
```

This indicates that the configuration contains an error.

Step 5: Restart the Service

Attempt to restart Apache.

```
sudo systemctl restart apache2
```

The restart operation will fail because of the invalid configuration.

Step 6: Verify Service Status

Check the service status again.

```
systemctl status apache2
```

Expected Output:

```
Active: failed (Result: exit-code)
```

Step 7: Display Failed Services

View all failed services on the system.

```
systemctl --failed
```

Sample Output:

UNIT	LOAD	ACTIVE	SUB	DESCRIPTION
apache2.service	loaded	failed	failed	The Apache HTTP Server

Step 8: Analyze Service Logs

View detailed logs for the failed service.

```
journalctl -xeu apache2
```

or

```
journalctl -u apache2 -n 20
```

Review the error messages and identify the root cause.

Step 9: Fix the Configuration

Open the configuration file again.

```
sudo nano /etc/apache2/apache2.conf
```

Remove the following line:

```
InvalidDirective On
```

Save and exit.

Step 10: Validate the Configuration Again

```
sudo apachectl configtest
```

Expected Output:

```
Syntax OK
```

Step 11: Restart the Service

```
sudo systemctl restart apache2
```

Verify:

```
systemctl status apache2
```

Expected Output:

```
Active: active (running)
```

Real-World Troubleshooting Workflow

Service Not Working



Check Status

(systemctl status)



Check Failed Services

(systemctl --failed)



Analyze Logs

(journalctl)



Identify Configuration Error



Fix the Error



Restart Service



Verify Service Status

Conclusion

The `systemctl` command is the primary tool used in Linux to manage system services. It allows administrators to start, stop, restart, enable, disable, monitor, and troubleshoot services efficiently. Understanding service management is a fundamental skill for Linux System Administrators, DevOps Engineers, and IT Infrastructure professionals.